

[LOGO ÉTABLISSEMENT]

[LOGO ENTREPRISE]

Rapport de Projet de Fin d'Études

Conception et déploiement d'une plateforme e-commerce en architecture microservices avec orchestration Kubernetes et pipeline GitOps

Réalisé par : [NOM PRÉNOM]

Établissement : [NOM ÉTABLISSEMENT]

Entreprise d'accueil : [NOM ENTREPRISE]

Encadrant pédagogique : [NOM ENCADRANT]

Maître de stage : [NOM MAÎTRE DE STAGE]

Année universitaire : [ANNÉE]

Sommaire

1. Introduction générale
2. Chapitre 1 — Présentation du contexte
 1. Présentation de l'organisme d'accueil
 2. Présentation du projet
 3. Environnement de travail
3. Chapitre 2 — Analyse et conception
 1. Analyse des besoins
 2. Architecture globale
 3. Conception de la base de données
 4. Conception des interfaces
4. Chapitre 3 — Réalisation et implémentation
 1. Environnement de développement
 2. Implémentation du backend
 3. Implémentation du frontend
 4. Fonctionnalités clés réalisées
 5. Sécurité et authentification
5. Chapitre 4 — Tests et validation
 1. Stratégie de tests
 2. Résultats des tests
 3. Recette fonctionnelle
6. Conclusion générale
7. Annexes

Introduction générale

L'évolution rapide du commerce électronique impose aux entreprises de disposer de plateformes logicielles à la fois robustes, évolutives et capables d'absorber des charges de trafic variables. Les architectures monolithiques traditionnelles, longtemps privilégiées pour leur simplicité de mise en œuvre, montrent leurs limites dès lors qu'il s'agit de faire évoluer indépendamment plusieurs domaines métiers, de déployer fréquemment de nouvelles fonctionnalités, ou de garantir une haute disponibilité.

Dans ce contexte, ce projet de fin d'études porte sur la **conception, le développement et le déploiement d'une plateforme e-commerce complète en architecture microservices**, avec une chaîne d'intégration et de livraison continues (CI/CD) entièrement automatisée selon les principes GitOps. Le système couvre l'ensemble du cycle de vie applicatif : du développement local à la mise en production sur un serveur dédié, en passant par les tests automatisés, la construction d'images Docker, et le déploiement déclaratif sur un cluster Kubernetes.

La **problématique** centrale traitée par ce projet est la suivante : *comment construire et déployer une plateforme e-commerce moderne en respectant les bonnes pratiques DevOps actuelles (microservices, conteneurisation, orchestration, observabilité, infrastructure-as-code), tout en garantissant un cycle de livraison rapide, traçable et réversible ?*

Les **objectifs** de ce rapport sont triples :

- Présenter de manière exhaustive l'architecture technique de la plateforme et justifier les choix technologiques effectués ;
- Détailler la stratégie de déploiement et le pipeline CI/CD mis en place ;
- Démontrer la maîtrise des outils cloud-native modernes (Kubernetes, ArgoCD, Sealed Secrets, Prometheus, Grafana).

Le présent rapport est organisé en **quatre chapitres**. Le premier chapitre présente le contexte du stage et les enjeux du projet. Le deuxième chapitre détaille l'analyse fonctionnelle et la conception. Le troisième chapitre décrit l'implémentation et les choix techniques. Enfin, le quatrième chapitre aborde la validation et les tests effectués. Une conclusion générale dresse le bilan du travail et présente les perspectives d'amélioration.

Chapitre 1 — Présentation du contexte

1.1 Présentation de l'organisme d'accueil

[ESPACE À COMPLÉTER]

Présentation de [NOM ENTREPRISE] : raison sociale, domaine d'activité, taille, organisation interne, secteur d'intervention.

Mentionner notamment le département au sein duquel s'est déroulé le stage ainsi que les missions habituelles de l'équipe technique d'accueil.

1.2 Présentation du projet

1.2.1 Objectifs

Le projet, baptisé **Ecommerce**, vise à concevoir une plateforme e-commerce complète selon les standards modernes du développement cloud-native. Au-delà des aspects fonctionnels classiques d'un site marchand (catalogue produits, panier, commande, paiement, livraison), le projet met l'accent sur l'*ingénierie de production* : conteneurisation, orchestration, observabilité, automatisation du déploiement.

Les objectifs détaillés sont :

- Implémenter une architecture orientée microservices découpée selon les domaines métier (Domain-Driven Design) ;
- Mettre en œuvre une communication asynchrone événementielle entre services via un broker de messages ;
- Construire un pipeline d'intégration continue automatisant les phases de qualité, de tests et de construction d'images ;
- Déployer la plateforme sur un cluster Kubernetes en suivant les pratiques GitOps (le dépôt Git comme source de vérité) ;
- Mettre en place une observabilité de production via collecte de métriques et tableaux de bord ;
- Sécuriser l'ensemble de la chaîne via HTTPS, secrets chiffrés et bonnes pratiques d'isolation.

1.2.2 Périmètre fonctionnel

La plateforme couvre les fonctionnalités suivantes :

- **Authentification et gestion des utilisateurs** : inscription, connexion, gestion des sessions, contrôle d'accès basé sur les rôles (RBAC) avec trois niveaux (`user`, `admin`, `editor`) ;

- **Catalogue produits** : consultation, recherche full-text via Redisearch, recommandations, gestion des avis clients ;
- **Panier d'achat et liste de souhaits** : stockage rapide en mémoire via Redis ;
- **Gestion des commandes et paiements** : intégration Stripe, traitement des transactions selon le pattern Saga ;
- **Logistique** : préparation des expéditions, suivi de livraison, gestion des coupons de réduction ;
- **Gestion des stocks** : suivi temps-réel des quantités, alertes de seuil bas ;
- **Tableau de bord administrateur** : gestion CRUD des produits, utilisateurs, commandes ;
- **Analyses et audit** : journalisation des actions sensibles, métriques d'usage, exports.

1.2.3 Enjeux techniques

Plusieurs enjeux techniques structurants ont guidé la conception :

- **Scalabilité horizontale** : chaque service doit pouvoir être répliqué indépendamment ;
- **Résilience** : la défaillance d'un composant ne doit pas compromettre l'ensemble du système ;
- **Déploiement continu** : tout changement de code validé doit pouvoir être déployé en production en moins de dix minutes ;
- **Sécurité** : les données sensibles (mots de passe, jetons, clés de paiement) doivent être protégées de bout en bout ;
- **Traçabilité** : chaque déploiement doit être réversible et associé à un commit Git précis.

1.3 Environnement de travail

Le projet a été mené selon une approche **itérative et incrémentale**, inspirée de la méthodologie Agile. Le développement s'est organisé en cycles courts, chaque itération aboutissant à un livrable fonctionnel testable. Le suivi de version a été assuré via **Git** avec un dépôt central hébergé sur **GitHub** (github.com/Montassur/microServiceApp).

Les outils utilisés pour la gestion et le développement comprennent :

- **Visual Studio Code** comme environnement de développement intégré ;
- **Docker Desktop** pour l'exécution locale des conteneurs ;
- **GitHub Actions** pour l'intégration continue ;
- **ArgoCD** pour le déploiement continu en mode GitOps ;
- **DBeaver** pour l'administration des bases de données PostgreSQL ;
- **OpenLens / k9s** pour l'exploration du cluster Kubernetes.

Chapitre 2 — Analyse et conception

2.1 Analyse des besoins

2.1.1 Besoins fonctionnels

L'analyse des sources du projet révèle les fonctionnalités majeures suivantes, regroupées par acteur :

Côté client (storefront) :

- F1 — Créer un compte utilisateur et se connecter ;
- F2 — Consulter le catalogue des produits, filtrer par catégorie, rechercher ;
- F3 — Consulter le détail d'un produit, lire les avis ;
- F4 — Ajouter des produits au panier, modifier les quantités ;
- F5 — Constituer une liste de souhaits ;
- F6 — Passer commande et payer en ligne ;
- F7 — Suivre l'état des commandes et le statut de livraison ;
- F8 — Recevoir des notifications applicatives ;
- F9 — Consulter et modifier son profil utilisateur.

Côté administrateur (admin dashboard) :

- F10 — Gérer le catalogue (CRUD produits) ;
- F11 — Gérer les utilisateurs (création, modification, désactivation) ;
- F12 — Consulter les commandes et leur état ;
- F13 — Gérer les coupons promotionnels ;
- F14 — Consulter les statistiques d'usage et les journaux d'audit.

2.1.2 Besoins non fonctionnels

Catégorie	Exigence
Performance	Temps de réponse API inférieur à 500 ms en p95
Disponibilité	Architecture conçue pour la haute disponibilité (2 répliques par service)
Scalabilité	Capacité à augmenter horizontalement chaque service indépendamment
Sécurité	Authentification JWT, HTTPS systématique, secrets chiffrés en dépôt Git
Observabilité	Collecte des métriques applicatives, tableaux de bord temps-réel
Maintenabilité	Code modulaire, configuration externalisée, tests automatisés
Portabilité	Déploiement identique en local (Docker Compose) et en production (Kubernetes)

2.2 Architecture globale

L'architecture retenue est une **architecture microservices à dix services**, orchestrée derrière une passerelle Nginx. Les services backend communiquent de manière synchrone via HTTP REST pour les requêtes utilisateur, et de manière asynchrone via le broker **RabbitMQ** pour les événements métier.

L'organisation en couches est la suivante :

- **Couche présentation** : deux applications React (storefront client et admin) servies par Nginx ;
- **Couche passerelle** : un reverse-proxy Nginx (`gateway`) qui route les requêtes vers le bon service backend selon le préfixe d'URL (`/api/auth` , `/api/products` , etc.) ;
- **Couche métier** : sept microservices Node.js Express, chacun gérant un domaine fonctionnel (authentification, catalogue, commandes, etc.) ;
- **Couche données** : PostgreSQL pour les données persistantes, Redis pour le cache et le moteur de recherche, RabbitMQ pour la messagerie événementielle.

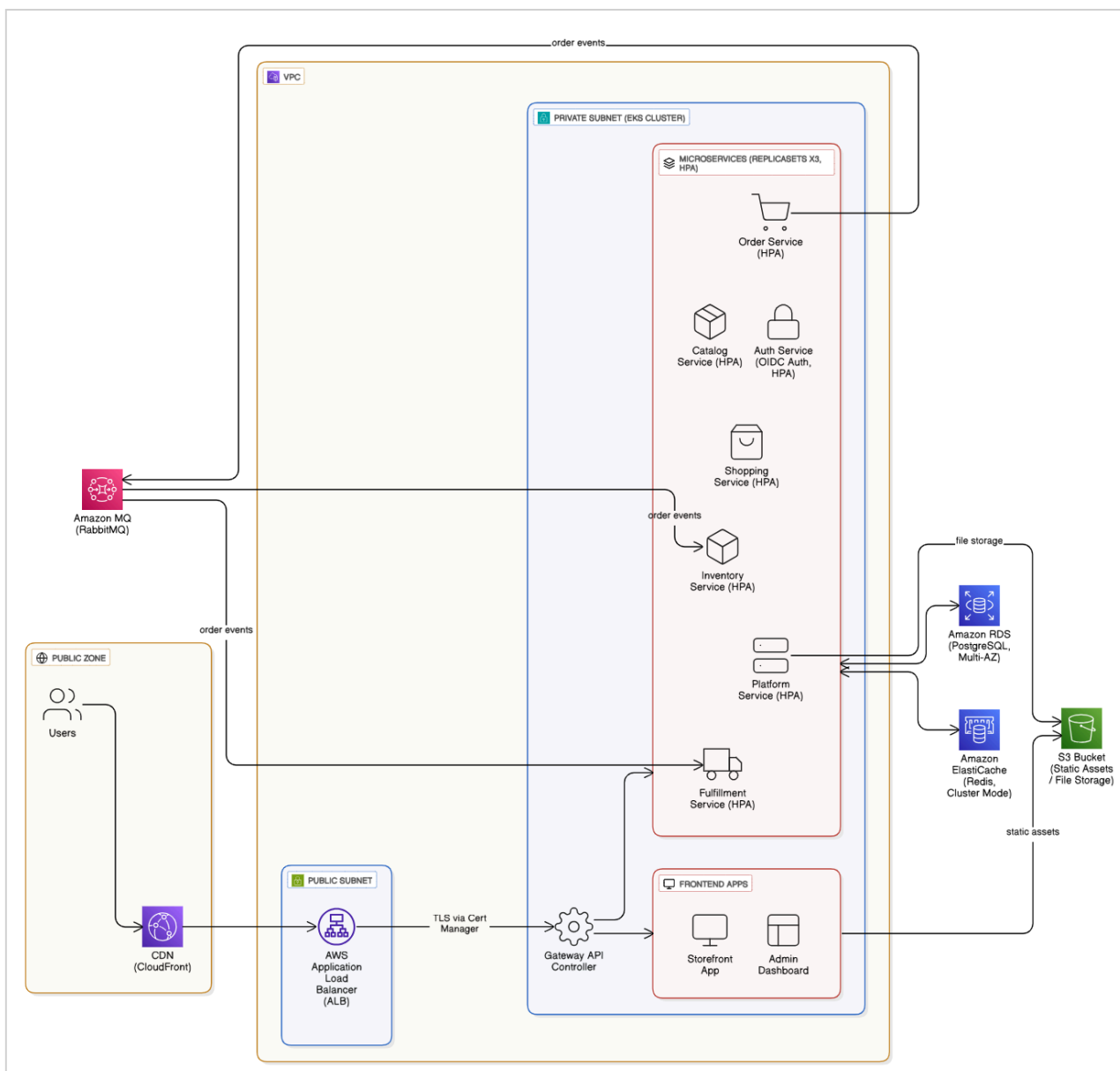


Figure 2.1 — Architecture globale de la plateforme en production (cluster Kubernetes k3s)

Choix technologiques justifiés

Couche	Technologie retenue	Justification
Frontend	React 19 + Vite	Écosystème mature, virtual DOM performant, build extrêmement rapide via Vite
API Gateway	Nginx	Reverse-proxy léger, configuration déclarative, performances éprouvées
Backend	Node.js 22 + Express	Adapté aux APIs I/O-bound, écosystème npm riche, courbe d'apprentissage faible
Base de données	PostgreSQL 17	SGBD relationnel robuste, support JSON natif, contraintes référentielles strictes
Cache / recherche	Redis Stack (Redisearch)	Cache en mémoire ultra-rapide, moteur de recherche full-text intégré
Messagerie	RabbitMQ 4.2	Broker AMQP éprouvé, garanties de livraison configurables, interface d'administration
Conteneurisation	Docker	Standard de facto pour la conteneurisation
Orchestration	Kubernetes (k3s)	Distribution allégée certifiée CNCF, idéale pour serveur unique
GitOps	ArgoCD	Synchronisation automatique cluster $\Leftrightarrow$ dépôt Git, interface graphique
Secrets	Sealed Secrets	Chiffrement par clé publique, secrets versionnés en Git en toute sécurité
Observabilité	Prometheus + Grafana	Standard de l'observabilité cloud-native
TLS	Let's Encrypt + Apache	Certificats gratuits, renouvellement automatique

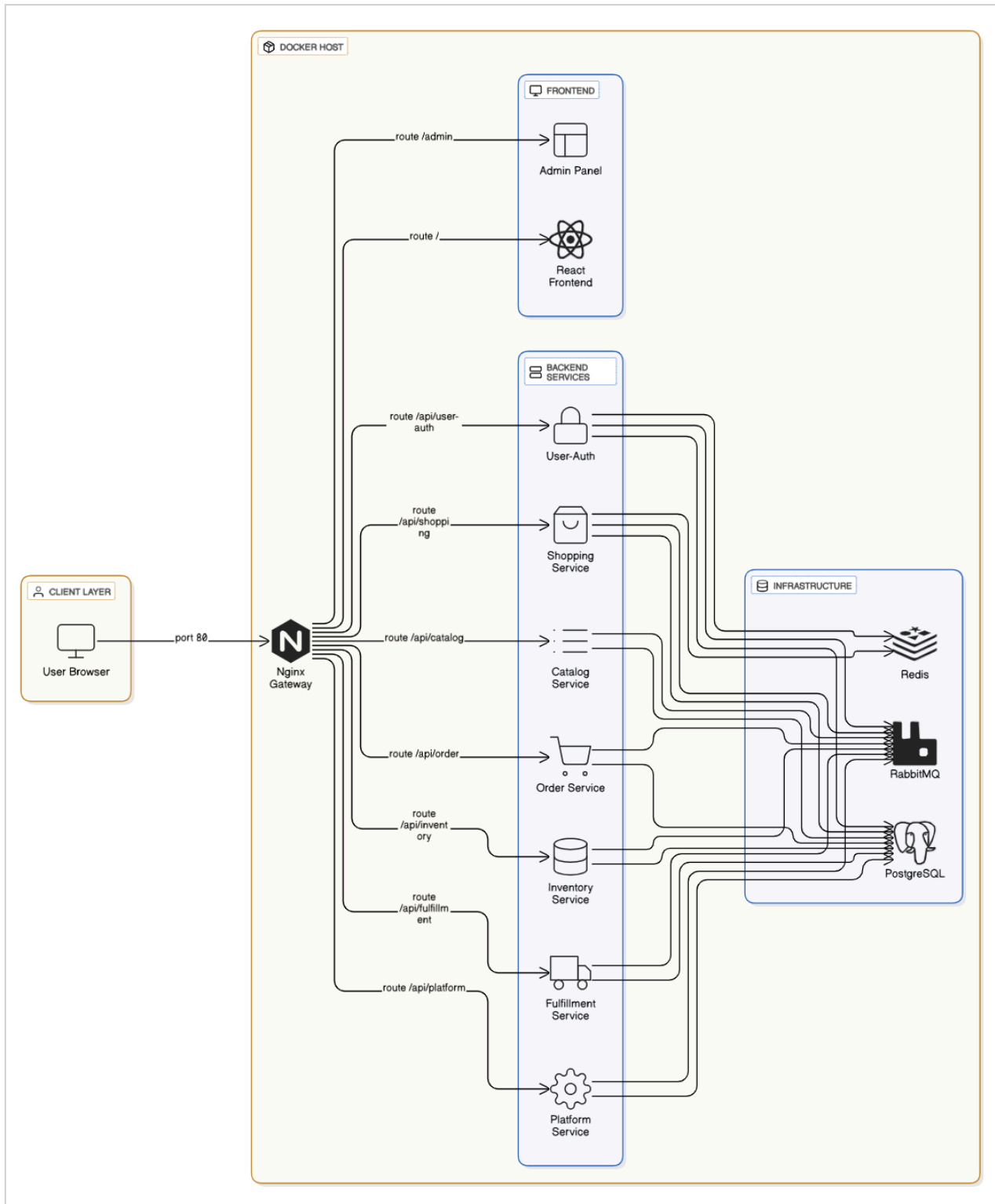


Figure 2.2 — Architecture de l'environnement de développement local (Docker Compose)

2.3 Conception de la base de données

Le schéma relationnel comporte **dix-sept tables** réparties selon les domaines fonctionnels des services. Les entités principales sont :

Table	Domaine	Rôle
users	user-auth	Comptes utilisateurs (email, mot de passe hashé, rôle, statut)
sessions	user-auth	Jetons de rafraîchissement, traçabilité d'IP
notifications	user-auth	Notifications applicatives
products	catalog	Catalogue produits
reviews	catalog	Avis utilisateurs
product_recommendations	catalog	Suggestions automatiques
orders	order-payment	Commandes
order_items	order-payment	Lignes de commande
payments	order-payment	Transactions Stripe
cart_items	shopping	Lignes du panier
wishlists	shopping	Liste de souhaits
inventory	inventory	Stocks par produit
shipments	fulfillment	Expéditions
coupons	fulfillment	Coupons promotionnels
audit_logs	platform	Audit des actions sensibles
analytics_events	platform	Métriques métier

La table `users` constitue la racine référentielle. Les sessions, notifications, commandes, paniers, listes de souhaits et logs y sont rattachés via clés étrangères avec contrainte `ON DELETE CASCADE`. Le schéma comprend plus de 25 index optimisant les requêtes par email, rôle, statut, catégorie produit et date.

[CAPTURE D'ÉCRAN : diagramme entité-association (ERD) généré depuis DBeaver]
Figure 2.3 — Modèle Entité-Association de la base PostgreSQL

2.4 Conception des interfaces

2.4.1 Storefront client

Le storefront est conçu autour des pages suivantes : page d'accueil (vitrine produit, recommandations), catalogue (grille produits avec filtres par catégorie), panier (récapitulatif, modification quantités, passage en caisse), commande (formulaire de paiement, confirmation), profil utilisateur (tableau de bord personnel, historique des commandes), authentification (inscription, connexion).

[CAPTURE D'ÉCRAN : maquette ou écran réel de la page d'accueil storefront]

Figure 2.4 — Page d'accueil du storefront client

2.4.2 Tableau de bord administrateur

L'interface admin propose un dashboard avec indicateurs clés (commandes, revenus, utilisateurs actifs), la gestion des utilisateurs (tableau filtrable, création, édition, désactivation, suppression par lot, export CSV), la gestion des produits, des coupons, et les paramètres généraux.

[CAPTURE D'ÉCRAN : dashboard administrateur — page de gestion des utilisateurs]

Figure 2.5 — Tableau de bord administrateur

Chapitre 3 — Réalisation et implémentation

3.1 Environnement de développement

3.1.1 Pile technique

Domaine	Technologie	Version
Runtime backend	Node.js	22-alpine
Framework backend	Express	4.x
Frontend	React	19.x
Build frontend	Vite	7.x
Base de données	PostgreSQL	17.2-alpine
Cache	Redis Stack	8.x
Broker messages	RabbitMQ	4.2-management-alpine
Reverse proxy	Nginx	1.28-alpine
Orchestrateur	Kubernetes (k3s)	v1.35
GitOps	ArgoCD	v2.x (stable)
Monitoring	Prometheus	v3.2.0
Visualisation	Grafana	11.5.0
Secrets chiffrés	Sealed Secrets	v0.27.1
Ingress	NGINX Ingress Controller	v1.11.3

3.1.2 Structure du projet

```

microServiceApp/
├── services/                                # 10 microservices
│   ├── frontend/                          # React storefront
│   ├── admin/                             # React admin
│   ├── gateway/                           # Nginx reverse-proxy
│   ├── user-auth/                         # auth + RBAC
│   ├── catalog/                           # produits + recherche
│   ├── order-payment/                     # commandes + Stripe
│   ├── fulfillment/                       # logistique
│   ├── shopping/                          # panier + wishlist
│   ├── platform/                          # audit, analytics
│   └── inventory/                         # gestion stocks
├── infrastructure/k8s/
│   ├── base/                              # Manifestes Kustomize de base
│   └── overlays/
│       ├── development/
│       └── production/                    # SealedSecret + Ingress + GHCR
├── database/
│   ├── init.sql                           # Schéma + 3 utilisateurs seed
│   └── seed_demo_data.sql
├── scripts/vps-deploy.sh                   # Déploiement automatisé VPS
├── argocd/applications.yaml                # ArgoCD Application
├── .github/workflows/ci.yml                # Pipeline CI/CD
├── docker-compose.yml
└── docs/                                  # 11 guides techniques

```

3.2 Implémentation du backend

3.2.1 Description des microservices

Chaque service backend respecte la même structure standardisée :

```

services/<nom>/
├── Dockerfile
├── package.json
└── src/
    ├── index.js                # Point d'entrée Express
    ├── lib/
    │   ├── db.js              # Pool PostgreSQL
    │   ├── redis.js           # Client Redis
    │   ├── rabbitmq.js        # Connexion AMQP
    │   ├── logger.js          # Winston
    │   └── metrics.js          # prom-client (Prometheus)
    ├── middleware/
    │   ├── auth.js
    │   └── metrics.js
    └── modules/<feature>/
        ├── routes.js          # Routes Express
        ├── init.js             # Création des tables
        └── events.js           # Publishers/consumers RabbitMQ

```

3.2.2 Endpoints API principaux

Service	Préfixe d'URL	Exemples d'endpoints
user-auth	/api/auth , /api/users	POST /login , POST /register , GET /me , POST /refresh
catalog	/api/products , /api/reviews , /api/search	CRUD produits, recherche full-text, recommandations
order-payment	/api/orders , /api/payments	POST commande, POST paiement Stripe
fulfillment	/api/shipping , /api/coupons	Suivi expédition, validation coupons
shopping	/api/cart , /api/wishlists	Manipulation panier et wishlist
platform	/api/analytics , /api/audit	Métriques, journaux d'audit, exports
inventory	/api/inventory	Consultation stocks par produit

3.2.3 Communication événementielle — Flux RabbitMQ

Les services communiquent de manière asynchrone via dix files d'événements. La figure ci-dessous illustre le flux principal d'une commande :

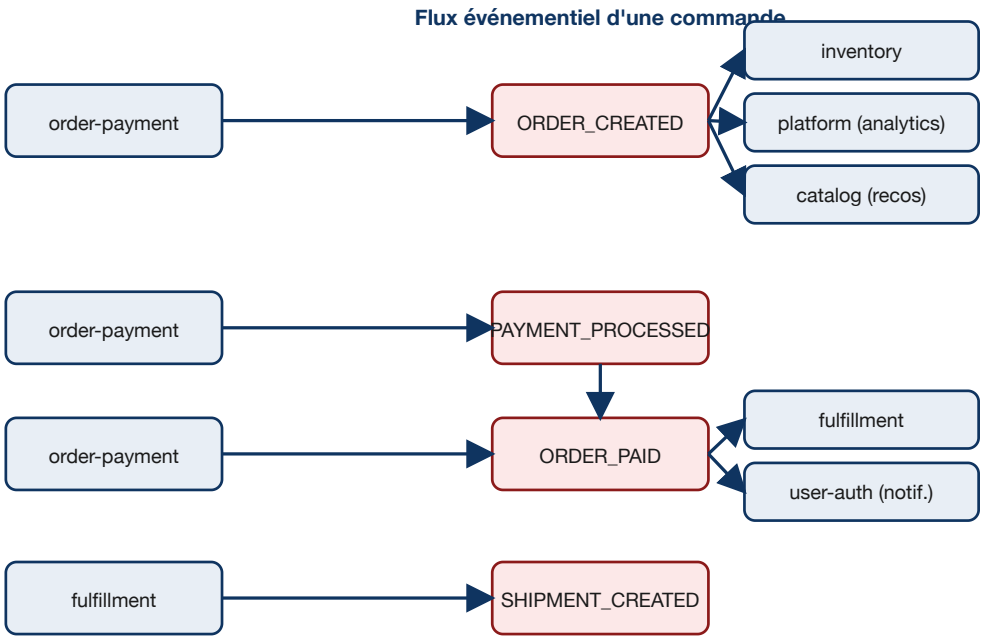


Figure 3.1 — Flux événementiel d'une commande client (pattern Saga via RabbitMQ)

Cette architecture événementielle implémente le pattern **Saga** : la suite ordonnée Cart → Order → Payment → Fulfillment est orchestrée par enchaînement d'événements, chaque service contribuant

à sa partie sans dépendance synchrone forte.

3.3 Implémentation du frontend

3.3.1 Storefront client

Le storefront est une **Single Page Application** React 19 + Vite, composée de pages (`Products` , `Cart` , `Orders` , `Profile` , `Dashboard` , `Login` , `Signup`), de contextes globaux (`AuthContext` , `CartContext`), et d'un client API basé sur Axios avec interception JWT et rafraîchissement automatique du token.

[CAPTURE D'ÉCRAN : page d'accueil storefront avec catalogue produits]

Figure 3.2 — Vue principale du storefront client

3.3.2 Dashboard administrateur

L'application admin propose des pages `Dashboard` , `Users` (avec recherche, filtres rôle/statut, suppression en masse, export CSV), `Products` , `Coupons` et `Settings` . Les notifications utilisateur sont gérées via un `ToastContext` dédié.

[CAPTURE D'ÉCRAN : page de gestion des utilisateurs (admin)]

Figure 3.3 — Interface de gestion des utilisateurs

3.4 Fonctionnalités clés réalisées

3.4.1 Pipeline CI/CD GitOps

Le cœur du projet est la chaîne CI/CD entièrement automatisée définie dans `.github/workflows/ci.yml` . Le schéma suivant en décrit le flux complet :

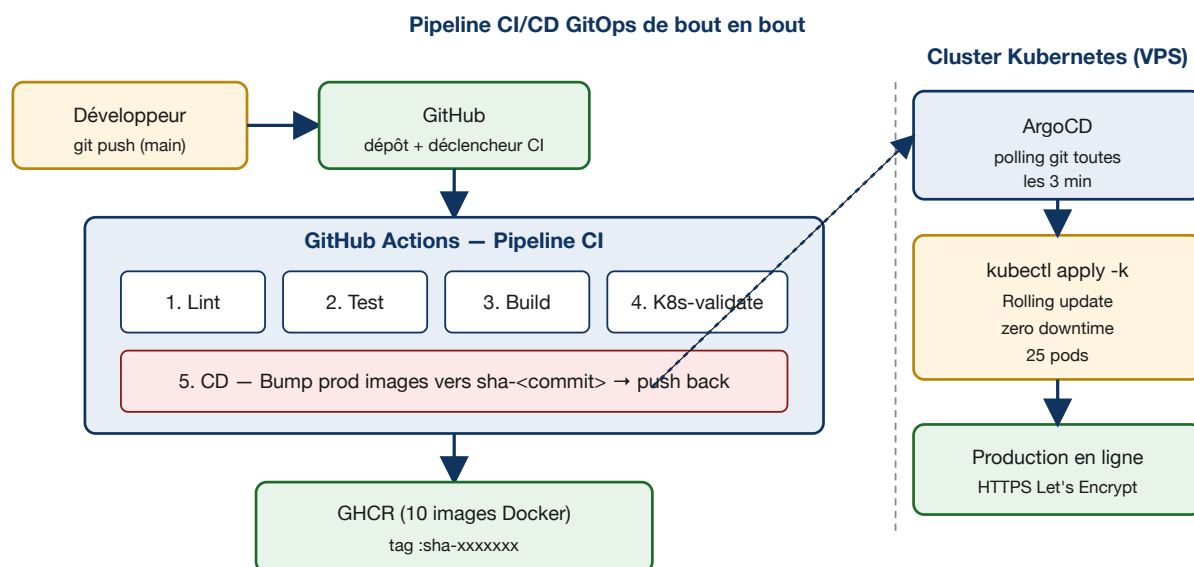


Figure 3.4 — Pipeline CI/CD complet : du `git push` à la production

Les cinq étapes du pipeline CI/CD sont :

1. **Lint** — exécution d'ESLint sur les 9 services applicatifs en parallèle ;
2. **Test** — exécution de Jest sur les 7 services backend ;
3. **Build** — construction Docker buildx et publication des 10 images sur GitHub Container Registry (GHCR), avec tags `branch` et `sha-<commit>` ;
4. **Validation Kubernetes** — `kubectl kustomize` sur les overlays développement et production ;
5. **CD** — **Bump des tags** — étape finale réécrivant `infrastructure/k8s/overlays/production/kustomization.yaml` avec le SHA du nouveau commit, puis poussant le changement avec le marqueur `[skip ci]` .

3.4.2 Recherche avancée via Redisearch

Le service `catalog` utilise le module **Redisearch** de Redis Stack pour offrir une recherche full-text instantanée. À chaque création/modification d'un produit, un événement RabbitMQ déclenche la mise à jour de l'index (`JSON.SET` , `FT.CREATE`).

3.4.3 Observabilité — Prometheus et Grafana

Chaque service backend expose un endpoint `/metrics` au format Prometheus. Le scrapping est effectué toutes les 15 secondes. Trois tableaux de bord Grafana sont provisionnés automatiquement :

- **Ecommerce Microservices Overview** : santé, CPU, mémoire, latence p95 par service ;
- **NodeJS Application Dashboard** : heap, event loop, garbage collector ;
- **RabbitMQ Overview** : files, consommateurs, taux de messages.

[CAPTURE D'ÉCRAN : tableau de bord Grafana — Ecommerce Microservices Overview]

Figure 3.5 — Tableau de bord Grafana en production

3.5 Sécurité et authentification

La sécurité a été abordée à plusieurs niveaux :

- **HTTPS systématique** via certificats Let's Encrypt renouvelés automatiquement par Apache + certbot ;
- **JWT signés** avec une clé aléatoire de 256 bits ;
- **Chiffrement des secrets** en dépôt Git via **Sealed Secrets** : seul le cluster possède la clé privée de déchiffrement ;
- **Isolation réseau** : NGINX Ingress n'expose que les services frontaux ;
- **Rate limiting** Nginx (zones `limit_req_zone`) contre les abus ;
- **En-têtes de sécurité** : X-Frame-Options, X-Content-Type-Options, Content-Security-Policy ;
- **Hashage bcrypt** à 10 itérations pour les mots de passe.

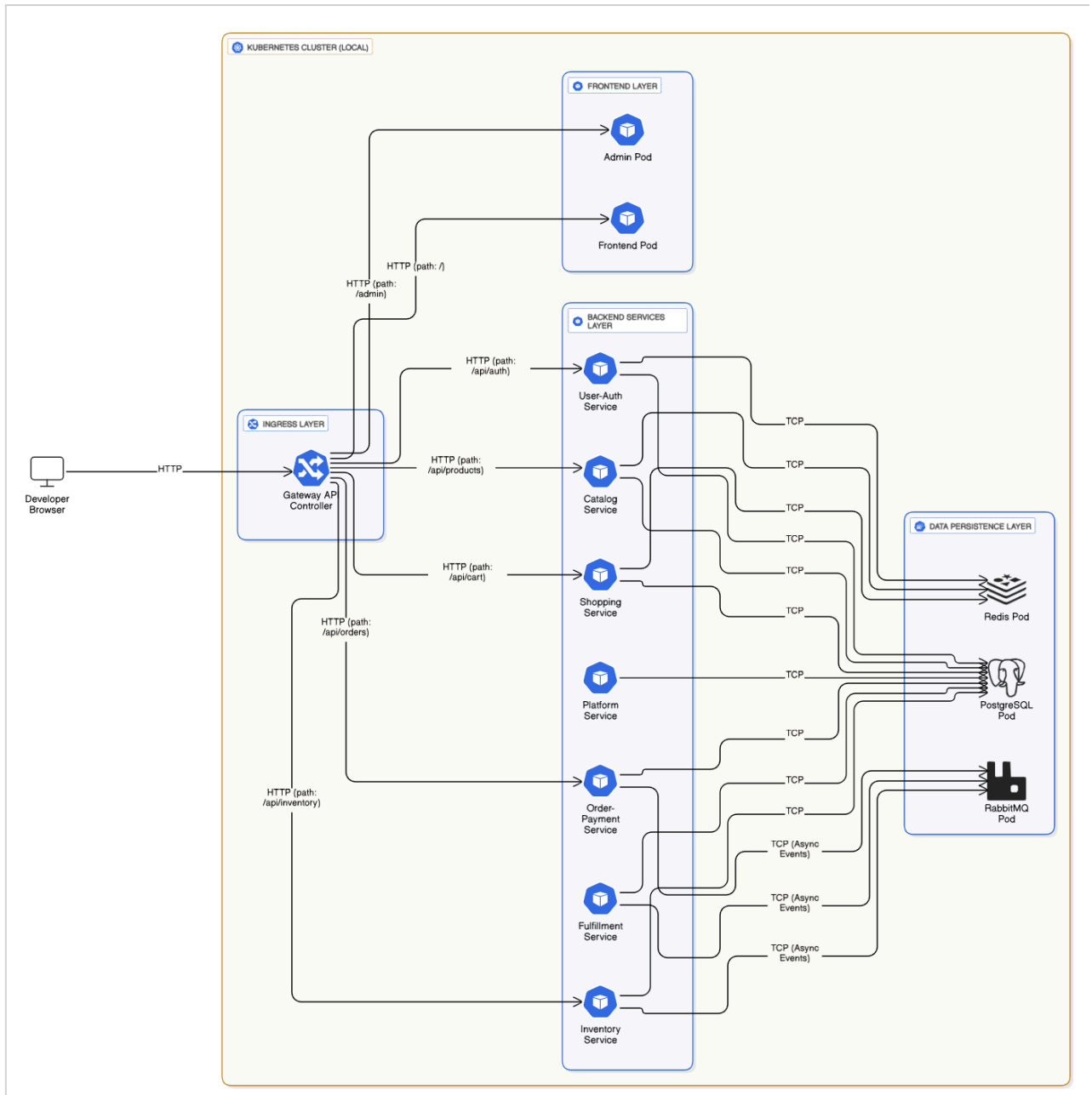


Figure 3.6 — Architecture de l'environnement de test K8s local (Docker Desktop)

Chapitre 4 — Tests et validation

4.1 Stratégie de tests

4.1.1 Qualité statique (Lint)

L'outil **ESLint** est exécuté automatiquement à chaque commit sur les 9 services disposant de code JavaScript/JSX. La configuration applique les règles strictes pour React 19 (`react-hooks/purity` , `react-hooks/exhaustive-deps` , `react-refresh/only-export-components`).

4.1.2 Tests unitaires

Le framework **Jest** est configuré sur les 7 services backend. Les scripts `npm test` sont prêts à recevoir des suites de tests dans le dossier `__tests__`/. À l'heure actuelle, la suite est volontairement vide (`--passWithNoTests` activé pour ne pas bloquer le CI), en attente d'évolution.

4.1.3 Tests d'intégration des manifestes Kubernetes

Le pipeline CI inclut une étape `k8s-validate` qui exécute `kubectl kustomize` sur les overlays développement et production, garantissant la validité syntaxique des manifestes et la résolution de toutes les références (ConfigMap, Secret, Service).

4.1.4 Tests de bout en bout (manuel)

La recette fonctionnelle a été conduite manuellement sur les flux utilisateurs critiques : inscription, connexion, navigation produit, ajout au panier, passage de commande, consultation des commandes par l'administrateur.

4.2 Résultats des tests

4.2.1 Pipeline CI

Job	Couverture	Résultat
Lint	9 services	9/9 OK
Test	7 services backend	7/7 OK (suite vide)
Build	10 images Docker	10/10 publiées sur GHCR
K8s-validate	2 overlays (dev + prod)	30 ressources générées
CD	Bump SHA prod	Commit auto poussé

[CAPTURE D'ÉCRAN : page GitHub Actions du dépôt montrant un pipeline vert]
Figure 4.1 — Pipeline CI/CD complet en succès

4.2.2 État du cluster de production

Namespace	Composants	Pods
ecommerce-prod	2 × 12 services applicatifs + 1 × postgres	25
monitoring	Prometheus + Grafana	2
argocd	Plateforme ArgoCD	7
ingress-nginx	Contrôleur Ingress	1
kube-system	Composants k3s	≈ 8

[CAPTURE D'ÉCRAN : sortie de `kubectl get pods -A` sur le cluster de production]
Figure 4.2 — État du cluster en production (25+ pods Running)

4.3 Recette fonctionnelle

Les cas d'usage suivants ont été validés sur l'environnement de production accessible publiquement via HTTPS :

Cas d'usage	URL testée	Résultat
Affichage storefront	<code>https://ecommerce.<domain></code>	HTTP 200
Inscription utilisateur	POST <code>/api/auth/register</code>	HTTP 201 + utilisateur en base
Connexion	POST <code>/api/auth/login</code>	HTTP 200 + JWT renvoyé
Consultation produits	GET <code>/api/products</code>	HTTP 200 + JSON
Dashboard admin	<code>https://admin.<domain></code>	HTTP 200, accès contrôlé
Interface ArgoCD	<code>https://argocd.<domain></code>	HTTP 200, synchronisation visible
Grafana	<code>https://grafana.<domain></code>	Tableaux de bord opérationnels
RabbitMQ Management	<code>https://rabbitmq.<domain></code>	10 files actives

[CAPTURE D'ÉCRAN : navigateur affichant le storefront en production avec HTTPS valide]

Figure 4.3 — La plateforme en production, accessible en HTTPS

Conclusion générale

Bilan du travail réalisé

Ce projet de fin d'études a permis de concevoir, développer et déployer une **plateforme e-commerce complète en architecture microservices** mise en production sur un cluster Kubernetes. Le système comprend dix microservices spécialisés, une chaîne d'intégration et de livraison continues entièrement automatisée selon les principes GitOps, ainsi qu'une couche d'observabilité opérationnelle (Prometheus, Grafana).

L'ensemble est accessible publiquement via six sous-domaines sécurisés en HTTPS, et tout changement de code validé est déployé automatiquement en production en moins de dix minutes. La séparation stricte entre l'environnement de développement local (Docker Compose) et la production (Kubernetes) garantit la sécurité de la chaîne de livraison.

Compétences acquises

Compétences techniques :

- Maîtrise de Node.js, Express et React 19 ;
- Conception et implémentation d'une architecture microservices ;
- Communication événementielle via RabbitMQ ;
- Conteneurisation Docker et orchestration Kubernetes (k3s) ;
- Mise en œuvre du modèle GitOps avec ArgoCD ;
- Sécurisation des secrets via Sealed Secrets ;
- Mise en place d'une chaîne CI/CD GitHub Actions ;
- Observabilité avec Prometheus et Grafana.

Compétences transversales :

- Diagnostic et résolution méthodique de problèmes complexes (lecture de logs, traçage inter-services) ;
- Documentation technique détaillée (PROJECT_GUIDE.md, RABBITMQ_EVENTS.md, SESSION_LOG.md) ;
- Capacité à raisonner sur des systèmes distribués.

Difficultés rencontrées et solutions

Difficulté	Solution apportée
Conflit de ports entre Apache (sites tiers) et NGINX Ingress	Mise en place de NGINX Ingress en NodePort 30080 et configuration d'Apache en reverse-proxy SSL
Schéma de base incohérent (colonne <code>name</code> manquante)	Ajout d'un <code>ALTER TABLE ... ADD COLUMN IF NOT EXISTS idempotent</code>
Cycle de dépendances Compose (frontend/admin/gateway)	Utilisation du résolveur DNS Docker (127.0.0.11) avec variables Nginx
Identifiants RabbitMQ incohérents K8s/Node	Construction de l'URL AMQP via substitution <code>\$(VAR)</code> Kubernetes
Synchronisation ArgoCD échouant sur Gateway API	Retrait des ressources Gateway/HTTPRoute de l'overlay production

Perspectives et améliorations futures

- **Suite de tests unitaires complète** : ajouter au moins un test de fumée par service (Jest + supertest) ;
- **Tests de charge** : intégrer k6 ou JMeter pour valider la tenue en charge ;
- **Documentation OpenAPI/Swagger** pour chaque API ;
- **Outil de migration de schéma** : intégrer node-pg-migrate ou Prisma Migrate ;
- **Tracing distribué** : déployer Jaeger ou Tempo avec instrumentation OpenTelemetry ;
- **Sauvegardes automatisées** : CronJob `pg_dump` → S3 ;
- **Mode multi-nœuds** : étendre k3s sur un second nœud pour la vraie HA ;
- **CDN devant la passerelle** : Cloudflare pour DDoS et cache statique.

Ce projet constitue une base solide et professionnelle, démontrant la maîtrise complète d'une chaîne de production cloud-native moderne — depuis la conception orientée domaines jusqu'au déploiement automatisé sur infrastructure réelle.

Annexes

Annexe A — Configuration ArgoCD

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ecommerce-production
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/Montassur/microServiceApp.git
    targetRevision: main
    path: infrastructure/k8s/overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: ecommerce-prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Annexe B — Glossaire technique

Terme	Définition
GitOps	Méthodologie où le dépôt Git est la source de vérité unique pour l'état souhaité du système. Tout changement passe par un commit.
Kustomize	Outil natif Kubernetes permettant de générer des variantes de manifestes via overlays sans templating.
ArgoCD	Contrôleur Kubernetes qui synchronise continuellement l'état du cluster avec le dépôt Git de référence.
Saga	Pattern de transaction distribuée par enchaînement d'événements compensables.
Sealed Secrets	Contrôleur qui déchiffre dans le cluster des secrets chiffrés versionnés en Git.
Ingress	Ressource Kubernetes qui expose des services HTTP/HTTPS depuis l'extérieur du cluster.
RBAC	Role-Based Access Control : contrôle d'accès basé sur les rôles assignés aux utilisateurs.
JWT	JSON Web Token : format de jeton signé pour l'authentification stateless.
k3s	Distribution Kubernetes légère certifiée CNCF, conçue pour les déploiements edge et single-node.
GHCR	GitHub Container Registry : registre d'images Docker hébergé par GitHub.
Pod	Plus petite unité déployable dans Kubernetes : regroupement de conteneurs partageant le réseau.
Deployment	Ressource Kubernetes qui gère le cycle de vie des Pods (mise à l'échelle, mise à jour progressive).
StatefulSet	Variante de Deployment pour les applications avec état (base de données, par exemple).

Annexe C — Références et ressources

- **Dépôt GitHub du projet** : github.com/Montassur/microServiceApp
- **Documentation Kubernetes** : kubernetes.io/docs
- **Documentation ArgoCD** : argo-cd.readthedocs.io
- **Documentation k3s** : docs.k3s.io
- **Bitnami Sealed Secrets** : github.com/bitnami-labs/sealed-secrets
- **Documentation Prometheus** : prometheus.io/docs

- **Documentation Grafana** : grafana.com/docs
- **Spécification AMQP 0.9.1** : rabbitmq.com/tutorials/amqp-concepts
- **Twelve-Factor App** : 12factor.net
- **Domain-Driven Design**, Eric Evans, 2003
- **Site Reliability Engineering**, Google, 2016